

JavaScript Notes

Chapter 6: Objects in JavaScript

1. Definition

An Object is a non-primitive data type used to store data in the form of key-value pairs. Each key is called a property, and the value can be any valid JavaScript data type, including another object or a function.

Example

```
const student = {  
  name: "Rahul",  
  age: 20,  
  course: "BCA"  
};
```

2. Why Do We Use Objects?

Objects help us to:

Store related data together.

Represent real-world entities.

Organize complex information.

Create reusable code.

Build scalable applications.

3. Creating Objects

Method 1: Object Literal (Recommended)

Syntax

```
const objectName = {  
  key1: value1,  
  key2: value2  
};
```

Example

```
const employee = {  
  name: "John",  
  age: 25,  
  salary: 50000  
};
```

```
console.log(employee);
```

Output

```
{
```

```
  name: "John",  
  age: 25,  
  salary: 50000  
}
```

Method 2: Using new Object()

Syntax

```
const obj = new Object();
```

Example

```
const car = new Object();
```

```
car.brand = "BMW";
```

```
car.model = "X5";
```

```
console.log(car);
```

4. Accessing Object Properties

There are two ways.

Dot Notation

Syntax

`object.property;`

Example

```
const student = {  
  name: "Rahul",  
  age: 20  
};
```

```
console.log(student.name);
```

Output

Rahul

Bracket Notation

Syntax

```
object["property"];
```

Example

```
console.log(student["age"]);
```

Output

20

When to use Bracket Notation?

Dynamic property names

Property names with spaces

Property names stored in variables

Example

```
const key = "name";
```

```
console.log(student[key]);
```

5. Adding New Properties

Example

```
const user = {  
  name: "Amit"  
};
```

```
user.age = 22;
```

```
console.log(user);
```

Output

```
{
```

```
name: "Amit",  
age: 22  
}
```

6. Updating Properties

Example

```
const user = {  
  age: 20  
};
```

```
user.age = 25;
```

```
console.log(user.age);
```

Output

```
25
```

7. Deleting Properties

Syntax

```
delete object.property;
```

Example

```
const user = {  
  name: "Rahul",  
  age: 20  
};
```

```
delete user.age;
```

```
console.log(user);
```

Output

```
{  
  name: "Rahul"  
}
```

8. Object Methods

Definition

When a function is stored inside an object, it is called an Object Method.

Example

```
const person = {  
  name: "John",  
  
  greet() {  
    console.log("Hello");  
  }  
}
```

```
};
```

```
person.greet();
```

Output

Hello

9. The this Keyword

Definition

this refers to the current object that is executing the method.

Example

```
const person = {  
  name: "Rahul",  
  
  greet() {  
    console.log(this.name);  
  }  
};
```

```
person.greet();
```

Output

Rahul

10. Nested Objects

Objects can contain other objects.

Example

```
const student = {  
  name: "Rahul",  
  
  address: {  
    city: "Delhi",  
    state: "Delhi"  
  }  
};
```

```
console.log(student.address.city);
```

Output

Delhi

11. Object Destructuring

Definition

Destructuring extracts properties from an object into variables.

Syntax

```
const { property } = object;
```

Example

```
const user = {  
  name: "Rahul",  
  age: 20  
};
```

```
const { name, age } = user;
```

```
console.log(name);  
console.log(age);
```

Output

```
Rahul  
20
```

12. Object.keys()

Definition

Returns an array containing all property names.

Example

```
const user = {
```

```
    name: "Rahul",  
    age: 20  
};
```

```
console.log(Object.keys(user));
```

Output

```
["name","age"]
```

13. Object.values()

Definition

Returns all values as an array.

Example

```
console.log(Object.values(user));
```

Output

```
["Rahul",20]
```

14. Object.entries()

Definition

Returns key-value pairs as nested arrays.

Example

```
console.log(Object.entries(user));
```

Output

```
[  
  ["name","Rahul"],  
  ["age",20]  
]
```

15. Object.assign()

Definition

Copies properties from one or more objects into another object.

Syntax

```
Object.assign(target, source);
```

Example

```
const obj1 = {  
  name: "Rahul"  
};
```

```
const obj2 = {  
  age: 20  
};
```

```
const result = Object.assign({}, obj1, obj2);
```

```
console.log(result);
```

Output

```
{  
  name:"Rahul",  
  age:20  
}
```

16. Object.freeze()

Definition

Prevents adding, deleting, or modifying properties.

Example

```
const user = {  
  name: "John"  
};
```

```
Object.freeze(user);
```

```
user.name = "Rahul";
```

```
console.log(user.name);
```

Output

John

17. Object.seal()

Definition

Prevents adding or deleting properties, but existing properties can still be modified.

Example

```
const user = {  
  name: "John"  
};
```

```
Object.seal(user);
```

```
user.name = "Rahul";
```

```
console.log(user.name);
```

Output

Rahul

18. Checking Property Existence

in Operator

```
const user = {  
  name: "Rahul"  
};
```

```
console.log("name" in user);
```

Output

true

hasOwnProperty()

```
console.log(user.hasOwnProperty("name"));
```

Output

true

19. Looping Through Objects

for...in Loop

Example

```
const user = {  
  name: "Rahul",  
  age: 20,  
  city: "Delhi"  
};  
  
for (let key in user) {  
  console.log(key, user[key]);  
}
```

Output

```
name Rahul  
age 20  
city Delhi
```

20. Difference Between Object and Array

Object	Array
--------	-------

Stores key-value pairs	Stores ordered values
------------------------	-----------------------

Uses property names	Uses indexes
---------------------	--------------

Best for structured data	Best for lists
--------------------------	----------------

Access using keys	Access using indexes
-------------------	----------------------

21. Features

Stores data as key-value pairs.

Can store any data type.

Supports nested objects.

Supports methods.

Dynamic and flexible.

Easily extensible.

22. Best Practices

Use const when the object reference doesn't change.

Use meaningful property names.

Prefer dot notation for known properties.

Use destructuring for cleaner code.

Keep related data together in one object.

23. Common Mistakes

Accessing a Missing Property

```
const user = {  
  name: "Rahul"  
};
```

```
console.log(user.email);
```

Output

undefined

Confusing Dot and Bracket Notation

 Incorrect

```
const key = "name";
```

```
console.log(user.key);
```

 Correct

```
console.log(user[key]);
```

24. Real-World Implementation

Objects are used in:

User profiles

Product details

Employee records

API responses (JSON)

Bank accounts

Student management systems

E-commerce products

Authentication systems

25. Interview Questions

Q1. What is an object?

An object is a collection of key-value pairs used to represent structured data.

Q2. What is the difference between dot notation and bracket notation?

Dot notation is used for fixed property names.

Bracket notation is used for dynamic or special property names.

Q3. What is the this keyword?

this refers to the current object that is executing the method.

Q4. What is object destructuring?

It extracts object properties into individual variables.

Q5. Difference between Object.freeze() and Object.seal()?

Object.freeze() prevents adding, deleting, and modifying properties.

Object.seal() prevents adding and deleting properties but allows modifying existing ones.

26. Key Points (Revision)

Objects store data as key-value pairs.

Access properties using dot (.) or bracket ([]) notation.

Use this to refer to the current object.

Object.keys() returns property names.

Object.values() returns property values.

Object.entries() returns key-value pairs.

Object.assign() copies or merges objects.

Object.freeze() makes an object immutable.

Object.seal() restricts adding/deleting properties.

for...in is used to iterate over object properties.

Next Chapter: Functions in JavaScript

We will cover:

Function Declaration

Function Expression

Arrow Functions

Anonymous Functions

Callback Functions

Higher-Order Functions

IIFE (Immediately Invoked Function Expression)

Default Parameters

Rest Parameters

Spread Operator

Return Statement

Scope

Hoisting

Closures

Recursion

Best Practices

Common Mistakes

Interview Questions

This is one of the most important JavaScript topics for interviews and real-world development.

JavaScript Notes

Chapter 7: Functions in JavaScript

1. Definition

A **Function** is a reusable block of code designed to perform a specific task. A function executes only when it is **called (invoked)**.

Think of a function as a machine: You give it input (parameters), it processes the input, and returns an output.

2. Why Do We Use Functions?

Functions help us to:

- Reuse code
 - Reduce code duplication
 - Improve readability
 - Make programs modular
 - Simplify debugging and maintenance
-

3. Function Declaration

Definition

A function declared using the `function` keyword is called a **Function Declaration**.

Syntax

```
function functionName(parameters) {  
    // code  
}
```

Example

```
function greet() {  
    console.log("Welcome to JavaScript");  
}  
  
greet();
```

Output

Welcome to JavaScript

Features

- Hoisted
 - Can be called before declaration
 - Reusable
-

4. Function Parameters & Arguments

Definition

- **Parameter:** Variable in the function definition.
- **Argument:** Actual value passed when calling the function.

Example

```
function greet(name) {  
    console.log("Hello " + name);  
}  
  
greet("Rahul");
```

Output

Hello Rahul

5. Function with Return Statement

Definition

The `return` keyword sends a value back to the caller and ends the function execution.

Syntax

```
function functionName() {  
    return value;  
}
```

Example

```
function add(a, b) {  
    return a + b;  
}  
  
let result = add(10, 20);
```

```
console.log(result);
```

Output

```
30
```

Key Points

- A function can return only one value directly.
- Code after `return` does not execute.

6. Function Expression

Definition

A function stored inside a variable is called a **Function Expression**.

Syntax

```
const variable = function() {  
  
};
```

Example

```
const greet = function () {  
    console.log("Hello");  
};  
  
greet();
```

Features

- Not hoisted like function declarations
- Can be anonymous

7. Anonymous Function

Definition

A function without a name is called an **Anonymous Function**.

Example

```
const message = function () {  
    console.log("Anonymous Function");  
};
```

```
message();
```

8. Arrow Function (ES6)

Definition

Arrow functions provide a shorter syntax for writing functions.

Syntax

```
const functionName = () => {  
  
};
```

Example

```
const greet = () => {  
  console.log("Hello");  
};  
  
greet();
```

Output

```
Hello
```

Arrow Function with Parameters

```
const add = (a, b) => {  
  return a + b;  
};  
  
console.log(add(10, 20));
```

Output

```
30
```

Short Arrow Function

```
const square = num => num * num;  
  
console.log(square(5));
```

Output

25

9. Default Parameters

Definition

Default parameters provide default values if no argument is passed.

Syntax

```
function greet(name = "Guest") {  
  
}
```

Example

```
function greet(name = "Guest") {  
    console.log(name);  
}  
  
greet();
```

Output

Guest

10. Rest Parameters (...)

Definition

Rest parameters collect multiple arguments into an array.

Syntax

```
function demo(...values) {  
  
}
```

Example

```
function total(...numbers) {  
  
    let sum = 0;  
  
    for(let num of numbers){
```

```
        sum += num;
    }

    return sum;
}

console.log(total(10,20,30));
```

Output

```
60
```

11. Spread Operator (...)

Definition

The spread operator expands an array or object into individual elements.

Example

```
let numbers = [10,20,30];

console.log(...numbers);
```

Output

```
10 20 30
```

12. Callback Function

Definition

A function passed as an argument to another function is called a **Callback Function**.

Example

```
function greet(name, callback){

    console.log("Hello " + name);

    callback();
}

function done(){
    console.log("Completed");
}
```

```
greet("Rahul", done);
```

Output

```
Hello Rahul  
Completed
```

Real-world Implementation

- API calls
- Event handling
- Timers
- Array methods

13. Higher-Order Function

Definition

A function that accepts another function as an argument or returns a function.

Example

```
function calculator(operation){  
  
    return operation(10,20);  
  
}  
  
function add(a,b){  
    return a+b;  
}  
  
console.log(calculator(add));
```

14. IIFE (Immediately Invoked Function Expression)

Definition

A function that runs immediately after it is created.

Syntax

```
(function(){  
  
    })();
```

Example

```
(function(){  
  
    console.log("Executed");  
  
})();
```

Output

```
Executed
```

15. Scope

Definition

Scope determines where variables can be accessed.

Types:

- Global Scope
- Function Scope
- Block Scope

Example

```
let a = 10;  
  
function test(){  
  
    let b = 20;  
  
    console.log(a);  
  
}  
  
test();
```

16. Hoisting

Definition

Hoisting is JavaScript's behavior of moving declarations to the top before execution.

Example

```
hello();

function hello(){
  console.log("Hello");
}
```

Output

```
Hello
```

17. Closure

Definition

A **Closure** is created when an inner function remembers variables from its outer function even after the outer function has finished executing.

Example

```
function outer(){

  let count = 0;

  return function(){

    count++;

    console.log(count);

  };

}

const counter = outer();

counter();
counter();
```

Output

```
1
2
3
```

Real-world Implementation

- Data privacy
 - Counters
 - Caching
 - Module pattern
-

18. Recursion

Definition

A function calling itself is called **Recursion**.

Example

```
function countdown(num){  
  
    if(num === 0){  
        return;  
    }  
  
    console.log(num);  
  
    countdown(num-1);  
  
}
```

```
countdown(5);
```

Output

```
5  
4  
3  
2  
1
```

19. Pure vs Impure Functions

Pure Function

Always returns the same output for the same input and has no side effects.

```
function add(a,b){  
  
    return a+b;
```

```
}
```

Impure Function

Depends on external data or changes external state.

```
let count = 0;

function increase(){

    count++;

}
```

20. Best Practices

- Use meaningful function names.
- Keep functions short and focused.
- Prefer `const` with arrow functions for modern JavaScript.
- Return values instead of using global variables.
- Avoid deeply nested functions unless closures are needed.

21. Common Mistakes

Forgetting `return`

✗ Incorrect

```
function add(a,b){
    a+b;
}

console.log(add(2,3));
```

Output

```
undefined
```

✓ Correct

```
function add(a,b){
    return a+b;
}
```

```
}
```

Calling Function Instead of Passing It

✗ Incorrect

```
button.addEventListener("click", greet());
```

✓ Correct

```
button.addEventListener("click", greet);
```

22. Real-World Implementation

Functions are used in:

- Form validation
- API requests
- Event handling
- Login systems
- Payment processing
- Calculations
- Authentication
- Shopping carts
- Games
- Dashboard applications

23. Interview Questions

Q1. What is a function?

A reusable block of code that performs a specific task.

Q2. Difference between Parameters and Arguments?

- **Parameters** are variables defined in the function.
- **Arguments** are the actual values passed during the function call.

Q3. Difference between Function Declaration and Function Expression?

- Function declarations are hoisted.
- Function expressions are not fully hoisted.

Q4. What is a Callback Function?

A function passed as an argument to another function and executed later.

Q5. What is a Closure?

A closure allows an inner function to access variables from its outer function even after the outer function has finished executing.

Q6. Difference between Rest and Spread Operators?

Rest (...)	Spread (...)
Collects multiple values into an array	Expands arrays or objects into individual elements
Used in function parameters	Used in function calls, arrays, and objects

24. Key Points (Revision)

- Functions make code reusable and modular.
- Function declarations are hoisted.
- Function expressions are stored in variables.
- Arrow functions provide a shorter syntax.
- `return` sends a value back to the caller.
- Callback functions are passed as arguments.
- Higher-order functions accept or return functions.
- Closures preserve access to outer scope variables.

- Recursion is a function calling itself.
 - Rest (`...`) collects values; Spread (`...`) expands values.
 - Pure functions have no side effects; impure functions do.
-

JavaScript Notes

Chapter 8: Function Methods (call(), apply(), bind())

1. Definition

In JavaScript, `call()`, `apply()`, and `bind()` are built-in methods used to **control the value of the `this` keyword** when invoking a function.

These methods are available on every function because functions are objects in JavaScript.

2. Why Do We Use call(), apply(), and bind()?

These methods help us to:

- Change the value of `this`
 - Borrow methods from another object
 - Reuse functions
 - Pass arguments in different ways
 - Improve code reusability
-

3. The `this` Keyword (Revision)

Definition

`this` refers to the object that is currently executing the function.

Example

```
const person = {
  name: "Rahul",

  greet() {
    console.log(this.name);
  }
};

person.greet();
```

Output

4. call() Method

Definition

The `call()` method invokes a function **immediately** and allows you to set the value of `this`. Arguments are passed **one by one**.

Syntax

```
functionName.call(thisArg, arg1, arg2, ...);
```

Example 1

```
const person = {  
  name: "Rahul"  
};  
  
function greet() {  
  console.log("Hello " + this.name);  
}  
  
greet.call(person);
```

Output

```
Hello Rahul
```

Example 2 (With Arguments)

```
const student = {  
  name: "Amit"  
};  
  
function introduce(age, city) {  
  console.log(this.name, age, city);  
}  
  
introduce.call(student, 21, "Delhi");
```

Output

```
Amit 21 Delhi
```

Features

- Invokes function immediately
 - Sets the `this` value
 - Arguments passed individually
-

5. apply() Method

Definition

The `apply()` method is similar to `call()`, but arguments are passed as an **array**.

Syntax

```
functionName.apply(thisArg, [arg1, arg2]);
```

Example

```
const employee = {
  name: "John"
};

function details(age, city) {
  console.log(this.name, age, city);
}

details.apply(employee, [25, "Mumbai"]);
```

Output

```
John 25 Mumbai
```

Features

- Invokes function immediately
 - Sets the `this` value
 - Arguments passed as an array
-

6. Difference Between call() and apply()

call()	apply()
Argument s are	Argument s are

passed individually	passed as an array
Executes immediately	Executes immediately
Better for fixed arguments	Better when arguments are already in an array

7. bind() Method

Definition

The `bind()` method **does not execute the function immediately**. Instead, it returns a **new function** with `this` permanently bound to the specified object.

Syntax

```
const newFunction = functionName.bind(thisArg);
```

Example

```
const user = {
  name: "Rahul"
};

function greet() {
  console.log("Hello " + this.name);
}

const newGreet = greet.bind(user);

newGreet();
```

Output

```
Hello Rahul
```

Example with Arguments

```
const person = {
  name: "John"
};

function intro(age) {
  console.log(this.name, age);
}

const show = intro.bind(person, 25);

show();
```

Output

```
John 25
```

Features

- Returns a new function
- Does not execute immediately
- Permanently binds `this`

8. Method Borrowing

Definition

Method Borrowing means using a method from one object inside another object using `call()`, `apply()`, or `bind()`.

Example

```
const person1 = {
  name: "Rahul",

  greet() {
    console.log("Hello " + this.name);
  }
};

const person2 = {
```

```
        name: "Amit"
    };

    person1.greet.call(person2);
```

Output

```
Hello Amit
```

9. Real-World Example

Using `bind()` in Event Listeners

```
const user = {
    name: "Rahul",

    greet() {
        console.log(this.name);
    }
};

const button = document.querySelector("button");

button.addEventListener("click", user.greet.bind(user));
```

Without `bind()`, `this` would refer to the button instead of the `user` object.

10. Best Practices

- Use `call()` when arguments are passed separately.
- Use `apply()` when arguments are already in an array.
- Use `bind()` for callbacks and event handlers.
- Avoid unnecessary binding.
- Understand how `this` changes in different contexts.

11. Common Mistakes

Forgetting `bind()` in Event Listeners

✗ Incorrect

```
button.addEventListener("click", user.greet);
```

`this` will not refer to `user`.

✓ Correct

```
button.addEventListener("click", user.greet.bind(user));
```

Using `bind()` Without Calling the Returned Function

✗ Incorrect

```
const greetUser = greet.bind(user);
```

Nothing happens because the new function is not executed.

✓ Correct

```
const greetUser = greet.bind(user);  
  
greetUser();
```

12. Real-World Implementation

`call()`, `apply()`, and `bind()` are used in:

- Event handling
 - React class components
 - Method borrowing
 - Callback functions
 - API libraries
 - Object-oriented programming
 - Utility/helper functions
-

13. Interview Questions

Q1. What is the purpose of `call()` ?

It invokes a function immediately and sets the value of `this`.

Q2. What is the difference between `call()` and `apply()` ?

- `call()` passes arguments individually.
 - `apply()` passes arguments as an array.
-

Q3. What is the difference between `call()` and `bind()` ?

- `call()` executes the function immediately.
- `bind()` returns a new function and does not execute it immediately.

Q4. What is Method Borrowing?

Using one object's method with another object by changing the `this` context using `call()`, `apply()`, or `bind()`.

Q5. Why is `bind()` commonly used with event listeners?

Because it ensures that `this` refers to the intended object instead of the DOM element that triggered the event.

14. Quick Comparison Table

Feature	<code>call()</code>	<code>apply()</code>	<code>bind()</code>
Executes Immediately	✓ Yes	✓ Yes	✗ No
Returns New Function	✗ No	✗ No	✓ Yes
Arguments	Individual	Array	Individual (or pre-bound)
Changes <code>this</code>	✓ Yes	✓ Yes	✓ Yes

15. Key Points (Revision)

- `call()` invokes the function immediately with a specified `this`.
- `apply()` is like `call()`, but accepts arguments as an array.
- `bind()` returns a new function with `this` permanently bound.
- All three methods are used to control the value of `this`.

- `bind()` is especially useful for callbacks and event listeners.
 - Method borrowing allows one object to use another object's methods without copying code.
-

Next Chapter: DOM (Document Object Model)

We will cover:

- What is the DOM?
- DOM Tree Structure
- Selecting Elements (`getElementById` , `querySelector` , etc.)
- Changing Content (`innerHTML` , `textContent`)
- Changing Styles
- Creating & Removing Elements
- Traversing the DOM
- DOM Manipulation
- Best Practices
- Common Mistakes
- Interview Questions

Note: DOM is one of the most important topics for JavaScript interviews and front-end development because it forms the foundation for building interactive web applications.

JavaScript Notes

Chapter 9: DOM (Document Object Model)

1. Definition

The DOM (Document Object Model) is a programming interface provided by the browser that represents an HTML document as a tree of objects. It allows JavaScript to access, modify, create, and delete HTML elements dynamically.

> Simple Definition:

The DOM is the bridge between HTML and JavaScript.

2. Why Do We Use DOM?

The DOM allows JavaScript to:

Access HTML elements

Change text and HTML content

Modify CSS styles

Handle user events

Create and remove elements

Build interactive web applications

3. DOM Tree Structure

Consider the following HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>DOM</title>
</head>
<body>

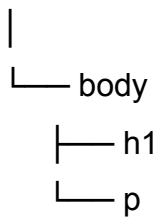
  <h1>Hello</h1>
  <p>Welcome</p>

</body>
</html>
```

DOM Tree

Document

```
|
└─ html
    |
    └─ head
        |
        └─ title
```

4. Selecting Elements

JavaScript provides several methods to select HTML elements.

A. getElementById()

Definition

Selects an element using its id.

Syntax

```
document.getElementById("idName");
```

Example

HTML

```
<h1 id="title">Hello</h1>
```

JavaScript

```
const title = document.getElementById("title");
```

```
console.log(title);
```

Features

Returns one element

Fastest selector

Uses unique id

B. `getElementsByClassName()`

Definition

Selects all elements having the same class.

Syntax

```
document.getElementsByClassName("className");
```

Example

```
<p class="text">One</p>
```

```
<p class="text">Two</p>
```

```
const items = document.getElementsByClassName("text");
```

```
console.log(items);
```

Features

Returns HTMLCollection

Live collection

C. `getElementsByTagName()`

Definition

Selects elements by tag name.

Syntax

```
document.getElementsByTagName("p");
```

D. `querySelector()`

Definition

Returns the first matching element.

Syntax

```
document.querySelector("selector");
```

Example

```
const heading = document.querySelector("h1");
```

E. querySelectorAll()

Definition

Returns all matching elements.

Syntax

```
document.querySelectorAll("selector");
```

Example

```
const paragraphs = document.querySelectorAll("p");
```

Features

Returns NodeList

Supports CSS selectors

5. Changing Content

innerHTML

Definition

Gets or sets HTML content.

Example

```
const heading = document.getElementById("title");
```

```
heading.innerHTML = "<i>JavaScript</i>";
```

Output

```
<i>JavaScript</i>
```

textContent

Definition

Gets or sets only text.

Example

```
heading.textContent = "Welcome";
```

Output

Welcome

innerText

Definition

Returns only visible text.

```
console.log(heading.innerText);
```

6. Changing CSS Styles

Syntax

```
element.style.property = value;
```

Example

```
const title = document.getElementById("title");
```

```
title.style.color = "red";
```

```
title.style.fontSize = "40px";
```

```
title.style.backgroundColor = "yellow";
```

7. Getting Attributes

getAttribute()

Definition

Returns the value of an attribute.

Example

```

```

```
const image = document.getElementById("image");
```

```
console.log(image.getAttribute("src"));
```

Output

cat.png

8. Setting Attributes

setAttribute()

Syntax

```
element.setAttribute(name,value);
```

Example

```
image.setAttribute("src", "dog.png");
```

9. Removing Attributes

```
image.removeAttribute("src");
```

10. Creating Elements

```
createElement()
```

Syntax

```
document.createElement("tagName");
```

Example

```
const heading = document.createElement("h2");
```

```
heading.textContent = "New Heading";
```

11. Adding Elements

```
appendChild()
```


Example

```
document.body.appendChild(heading);
```

`append()`

Adds one or more nodes.

```
document.body.append(heading);
```

`prepend()`

Adds an element at the beginning.

```
document.body.prepend(heading);
```

12. Removing Elements

`remove()`

```
heading.remove();
```

`removeChild()`

`parent.removeChild(child);`

13. Traversing the DOM

`parentElement`

`element.parentElement;`

`children`

`element.children;`

`firstElementChild`

`element.firstElementChild;`

`lastElementChild`

`element.lastElementChild;`

`nextElementSibling`

`element.nextElementSibling;`

`previousElementSibling`

`element.previousElementSibling;`

14. `classList`

`add()`

`element.classList.add("active");`

`remove()`

`element.classList.remove("active");`

`toggle()`

`element.classList.toggle("dark");`

`contains()`

`console.log(element.classList.contains("active"));`

15. Difference Between HTMLCollection and NodeList

HTMLCollection	NodeList
----------------	----------

Live collection	Static collection
-----------------	-------------------

Returned by <code>getElementsBy...</code>	Returned by <code>querySelectorAll()</code>
---	---

Updates automatically	Does not update automatically
-----------------------	-------------------------------

16. Best Practices

Use `querySelector()` for single elements.

Use `querySelectorAll()` for multiple elements.

Prefer `textContent` when adding plain text.

Avoid excessive use of `innerHTML` because it can introduce security risks if used with untrusted input.

Use `classList` instead of directly changing many style properties.

17. Common Mistakes

Accessing an Element Before It Exists

 Incorrect

```
const btn = document.getElementById("btn");
```

If the script runs before the HTML is loaded, `btn` will be `null`.

 Correct

```
<body>
```

```
<button id="btn">Click</button>
```

```
<script src="script.js"></script>
```

```
</body>
```

Or use:

```
document.addEventListener("DOMContentLoaded", () => {  
    const btn = document.getElementById("btn");  
});
```

Using innerHTML for Plain Text



```
element.innerHTML = userInput;
```



```
element.textContent = userInput;
```

18. Real-World Implementation

DOM is used in:

Login forms

Registration forms

Todo applications

Shopping carts

Image sliders

Navigation menus

Dark mode toggle

Chat applications

Dashboards

E-commerce websites

19. Interview Questions

Q1. What is the DOM?

The DOM is a tree-like representation of an HTML document that allows JavaScript to access and manipulate web page elements.

Q2. Difference between innerHTML and textContent?

innerHTML reads or writes HTML content.

textContent reads or writes only plain text.

Q3. Difference between `querySelector()` and `querySelectorAll()`?

`querySelector()` returns the first matching element.

`querySelectorAll()` returns all matching elements as a `NodeList`.

Q4. Difference between `HTMLCollection` and `NodeList`?

`HTMLCollection` is live.

`NodeList` from `querySelectorAll()` is static.

Q5. What is `classList` used for?

It is used to add, remove, toggle, and check CSS classes on an element.

20. Key Points (Revision)

The DOM connects HTML and JavaScript.

Use `getElementById()` to select elements by ID.

Use `querySelector()` for the first matching CSS selector.

Use `querySelectorAll()` for all matching elements.

`innerHTML` handles HTML content; `textContent` handles plain text.

Use `createElement()` and `appendChild()` to create and insert elements.

Use `remove()` to delete elements.

`classList` provides methods to manage CSS classes.

DOM manipulation is essential for creating interactive web applications.

Next Chapter: Events in JavaScript

We will cover:

What are Events?

Event Listeners

Mouse Events

Keyboard Events

Form Events

Window Events

Event Object

Event Bubbling

Event Capturing

Event Delegation

Prevent Default

Stop Propagation

Best Practices

Common Mistakes

Interview Questions

> Events + DOM are the foundation of interactive websites and are among the most frequently asked topics in JavaScript interviews.

JavaScript Notes

Chapter 10: Events in JavaScript

1. Definition

An **Event** is an action or occurrence that happens in the browser, such as a user clicking a button, pressing a key, submitting a form, or moving the mouse. JavaScript can detect these events and execute code in response.

Simple Definition:

An event is something that happens on a webpage, and JavaScript reacts to it.

2. Why Do We Use Events?

Events make websites interactive. They help us to:

- Respond to user actions
 - Validate forms
 - Build menus
 - Create games
 - Handle API requests
 - Improve user experience
-

3. Event Flow

Whenever an event occurs:

```
graph TD; A[User Action] --> B[Browser Detects Event]; B --> C[JavaScript Event Listener]; C --> D[Function Executes];
```

The diagram illustrates the event flow process. It starts with a 'User Action', which leads to the 'Browser Detects Event'. This then triggers the 'JavaScript Event Listener', which finally results in the 'Function Executes'.

4. Ways to Add Events

A. Inline Event (Not Recommended)

HTML

```
<button onclick="showMessage()">Click Me</button>
```

JavaScript

```
function showMessage() {  
    alert("Button Clicked");  
}
```

Features

- Easy to write
- Not recommended for large projects

B. Using Event Property

Syntax

```
element.onclick = function () {  
  
};
```

Example

```
const btn = document.getElementById("btn");  
  
btn.onclick = function () {  
    console.log("Clicked");  
};
```

C. Using addEventListener() (Recommended)

Definition

Adds one or more event listeners to an element without overwriting existing ones.

Syntax

```
element.addEventListener("event", callback);
```

Example

```
const btn = document.getElementById("btn");  
  
btn.addEventListener("click", function () {  
    console.log("Button Clicked");  
});
```

Features

- Multiple listeners allowed
 - Cleaner code
 - Most commonly used
-

5. Mouse Events

Event	Description
click	Mouse click
dblclick	Double click
mousedown	Mouse button pressed
mouseup	Mouse button released
mousemove	Mouse moves
mouseenter	Mouse enters element
mouseleave	Mouse leaves element
contextmenu	Right-click

Example

```
button.addEventListener("click", () => {  
    console.log("Button Clicked");  
});
```

6. Keyboard Events

Event	Description
keydown	Key is pressed
keyup	Key is released
keypress	Key pressed (deprecated)

Example

```
document.addEventListener("keydown", (event) => {  
    console.log(event.key);  
});
```

Output

```
a  
Enter  
ArrowUp
```

7. Form Events

Event	Description
submit	Form submitted
change	Value changed
input	User types
focus	Input gains focus
blur	Input loses focus
reset	Form reset

Example

```
const form = document.querySelector("form");

form.addEventListener("submit", function () {
  console.log("Form Submitted");
});
```

8. Window Events

Event	Description
-------	-------------

load	Page loaded
resize	Window resized
scroll	Page scrolled
unload	Page closed

Example

```
window.addEventListener("load", () => {  
    console.log("Website Loaded");  
});
```

9. Event Object

Definition

Whenever an event occurs, JavaScript automatically creates an **Event Object** containing information about that event.

Syntax

```
element.addEventListener("click", function(event){  
  
});
```

Example

```
button.addEventListener("click", function(event){  
  
    console.log(event);  
  
});
```

Common Properties

Property	Description
target	Element that triggered the event
type	Event type
key	Pressed key
clientX	Mouse X position
clientY	Mouse Y position

10. preventDefault()

Definition

Stops the browser's default behavior.

Example

```
const form = document.querySelector("form");

form.addEventListener("submit", function(event){

    event.preventDefault();

    console.log("Form Submitted");

});
```

Real-world Use

- Form validation

- Prevent page reload
 - Custom navigation
-

11. stopPropagation()

Definition

Stops an event from propagating to parent elements.

Example

```
child.addEventListener("click", function(event){  
  
    event.stopPropagation();  
  
});
```

12. Event Bubbling

Definition

In **Event Bubbling**, an event starts from the target element and moves upward to its parent elements.

Example

```
<div id="parent">  
    <button id="child">Click</button>  
</div>
```

```
parent.addEventListener("click", () => {  
    console.log("Parent");  
});  
  
child.addEventListener("click", () => {  
    console.log("Child");  
});
```

Output

```
Child  
Parent
```

13. Event Capturing

Definition

In **Event Capturing**, the event travels from the outermost parent down to the target element.

Syntax

```
element.addEventListener("click", callback, true);
```

Example

```
parent.addEventListener("click", () => {  
    console.log("Parent");  
}, true);
```

14. Event Delegation

Definition

Event Delegation is a technique where a single parent element handles events for its child elements using event bubbling.

Example

```
document.querySelector("ul").addEventListener("click", (event) => {  
  
    if (event.target.tagName === "LI") {  
        console.log(event.target.textContent);  
    }  
  
});
```

Advantages

- Better performance
- Less memory usage
- Handles dynamically added elements

15. removeEventListener()

Definition

Removes an event listener.

Example

```
function greet() {  
    console.log("Hello");  
}
```

```
}

button.addEventListener("click", greet);

button.removeEventListener("click", greet);
```

16. Best Practices

- Use `addEventListener()` instead of inline events.
- Keep event handler functions small.
- Use `preventDefault()` only when necessary.
- Remove unused event listeners.
- Use event delegation for large lists.

17. Common Mistakes

Forgetting the Event Object

✗ Incorrect

```
button.addEventListener("click", function(){

    console.log(event.target);

});
```

`event` may not be available in all environments.

✓ Correct

```
button.addEventListener("click", function(event){

    console.log(event.target);

});
```

Calling the Function Instead of Passing It

✗ Incorrect

```
button.addEventListener("click", greet());
```

✓ Correct

```
button.addEventListener("click", greet);
```

18. Real-World Implementation

Events are used in:

- Login buttons
- Registration forms
- Search bars
- Navigation menus
- Image sliders
- Shopping carts
- Games
- Chat applications
- Drag-and-drop features
- Infinite scrolling

19. Interview Questions

Q1. What is an event in JavaScript?

An event is an action or occurrence (such as a click or key press) that JavaScript can detect and respond to.

Q2. Why is `addEventListener()` preferred?

Because it allows multiple event listeners on the same element and keeps JavaScript separate from HTML.

Q3. What is the difference between Event Bubbling and Event Capturing?

Event Bubbling	Event Capturing
Child → Parent	Parent → Child

Default behavior	Enabled with <code>true</code> in <code>addEventListener()</code>
------------------	---

Q4. What does `preventDefault()` do?

It prevents the browser's default action, such as submitting a form or following a link.

Q5. What is Event Delegation?

A technique where a parent element handles events for its child elements using event bubbling, improving performance and simplifying event management.

20. Key Points (Revision)

- Events make web pages interactive.
 - `addEventListener()` is the recommended way to handle events.
 - Mouse, keyboard, form, and window events are the most common categories.
 - The **Event Object** provides information about the event.
 - `preventDefault()` stops the browser's default behavior.
 - `stopPropagation()` prevents an event from bubbling up.
 - Event Bubbling flows **Child** → **Parent**.
 - Event Capturing flows **Parent** → **Child**.
 - Event Delegation improves performance by handling child events through a parent element.
-

Next Chapter: Control Statements in JavaScript

We'll cover:

- `if`
- `if...else`
- `else if`
- Nested `if`

- `switch`
- Ternary Operator
- Truthy & Falsy Values
- Short-Circuit Evaluation
- Best Practices
- Common Mistakes
- Interview Questions

This chapter is essential because control statements determine the flow of execution in JavaScript programs.